`dispatch`, the related binding would be named `dispatch-in-0` and `dispatch-out-0`. We actually used a composed function (`pack|label`), so the binding names are generated by combining the names of all the functions involved in the composition:

- Input binding: `packlabel-in-0`
- Output binding: `packlabel-out-0`

These names are only relevant for configuring the bindings themselves in the application. They're like unique identifiers that let you reference a specific binding and apply custom configuration. Notice that these names exist only in Spring Cloud Stream—they're logical names. RabbitMQ doesn't know about them.

CONFIGURING DESTINATION BINDINGS

By default, Spring Cloud Stream uses the binding names to generate the names for exchanges and queues in RabbitMQ, but in a production scenario you'd probably want to manage them explicitly for several reasons. For example, it's likely that both exchanges and queues already exist in production. You will also want to control different options for exchanges and queues, like durability or routing algorithms.

For Dispatcher Service, I'll show you how to configure input and output bindings. At startup, Spring Cloud Stream will check if the related exchanges and queues already exist in RabbitMQ. If they don't, it will create them according to your configuration.

Let's start by defining the destination names that will be used to name exchanges and queues in RabbitMQ. In your Dispatcher Service project, update the application.yml file as follows.